

Mockito Guide

1. What is Mockito

Mockito is a Java mocking framework used to create test doubles for dependencies such as repositories, REST clients, file readers, and network clients. Mocking lets a class be tested in isolation, without needing a real database, a live API, or an actual file system.

2. Core Mockito API

Call	Purpose
<code>mock()</code>	Creates a fake object.
<code>when()</code>	Defines what the fake object should return.
<code>verify()</code>	Checks whether a method was called.
<code>doThrow()</code>	Makes a void method throw an exception.
<code>times()</code>	Checks how many times a method was called.
<code>anyString()</code>	Matches any String argument.
<code>inOrder()</code>	Verifies method call order.

3. Basic Mocking Exercises (package: basic)

MockingTest — Mocking and stubbing an external API with a plain Mockito mock.

VerifyTest — Verifying that a mocked method was called using `verify()`.

ArgumentMatcherTest — Stubbing and verifying a call using `anyString()` as an argument matcher.

VoidMethodTest — Stubbing a void method with `doNothing()` and verifying it ran.

MultipleReturnTest — Chaining `thenReturn()` to return different values on consecutive calls.

OrderVerificationTest — Using `InOrder` to check that two mocked calls happened in sequence.

ExceptionTest — Using `doThrow()` to make a void method throw an exception, verified with `assertThrows()`.

4. Spring Test Mocking Exercises (package: spring)

UserServiceTest — Mocks `UserRepository` directly with Mockito and tests `UserService` in isolation.

UserControllerTest — Uses `@WebMvcTest` with `MockMvc` and `@MockBean` to test only the web layer of `UserController`.

UserIntegrationTest — Uses `@SpringBootTest` with `@AutoConfigureMockMvc`, mocking `UserService` with `@MockBean` while the rest of the context loads.

Note: `@MockBean` replaces a bean in the Spring context with a Mockito mock, which is different from a plain `mock()` call used outside a Spring context.

5. Advanced Mocking Exercises (package: advanced)

DatabaseMockTest — Mocks a generic Repository and tests DataService, which wraps the repository's data.

RestApiMockTest — Mocks a RestClient and tests ApiService, which formats the REST response.

FileMockTest — Mocks a FileReader and FileWriter and tests FileService, which reads and processes file content.

NetworkMockTest — Mocks a NetworkClient and tests NetworkService, which formats the connection result.

MultiReturnServiceTest — Mocks a Repository to return two different values on consecutive calls, verified through DataService.

6. Running the Tests

Run everything: `mvn test`

Run a single class: `mvn -Dtest=MockingTest test`

Run everything in one package: `mvn -Dtest=com.example.mockito.basic.* test`